

**SIMATS ENGINEERING**  
**ASSESSMENT TOOL 2: Schema Analysis and Viva Voce**

**COURSE NAME:** Query Processing for Data Science    **COURSE CODE:** DSA0503

---

**COURSE OUTCOME COVERED**

**CO2:** Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BTL6)

Assessment Tool Weightage: 35%

Dave's Taxonomy: Manipulation (Level 2)    **Total Marks: 20**

Question Count: 2

---

**Code of Conduct**

I \_\_\_\_\_ S. V. K. Sharon Surya(192324246)\_\_\_\_(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: \_\_\_\_\_

**Database Design and Connectivity Rubric**

| Requirement Analysis<br>(4 Marks) | ER Diagram Design<br>(4 Marks) | Table Design & Normalization<br>(4 Marks) | Database Connectivity using Python (4 Marks) | CRUD Operations<br>(4 Marks) | Total<br>(20 Marks) |
|-----------------------------------|--------------------------------|-------------------------------------------|----------------------------------------------|------------------------------|---------------------|
|                                   |                                |                                           |                                              |                              |                     |

**Total Marks Awarded:**

## Question 1: Student Course Registration Database

**Aim**

To connect to an SQLite database and retrieve the names of students along with the courses they enrolled in during the **Odd 2026** semester using SQL JOIN.

## Theory

SQLite is a lightweight relational database. SQL JOIN is used to combine records from multiple tables based on related columns.

### CODE:

```
import sqlite3

conn = sqlite3.connect("university.db")
cur = conn.cursor()

# Create tables
cur.execute("""
CREATE TABLE IF NOT EXISTS Student(
Student_ID INTEGER PRIMARY KEY,
Student_Name TEXT,
Department TEXT,
Year INTEGER)
""")

cur.execute("""
CREATE TABLE IF NOT EXISTS Course(
Course_ID INTEGER PRIMARY KEY,
Course_Name TEXT,
Credits INTEGER)
""")

cur.execute("""
CREATE TABLE IF NOT EXISTS Enrollment(
Enrollment_ID INTEGER PRIMARY KEY,
Student_ID INTEGER,
Course_ID INTEGER,
```

```
Semester TEXT,  
FOREIGN KEY(Student_ID) REFERENCES Student(Student_ID),  
FOREIGN KEY(Course_ID) REFERENCES Course(Course_ID))  
""")
```

```
# Insert data
```

```
cur.execute("DELETE FROM Student")  
cur.execute("DELETE FROM Course")  
cur.execute("DELETE FROM Enrollment")
```

```
cur.executemany("INSERT INTO Student VALUES(?,?,?)",[  
(1,'Rahul','CSE',2),  
(2,'Anitha','AI',3),  
(3,'Kiran','ECE',1)  
])
```

```
cur.executemany("INSERT INTO Course VALUES(?,?,?)",[  
(101,'Python',4),  
(102,'DBMS',3),  
(103,'AI',4)  
])
```

```
cur.executemany("INSERT INTO Enrollment VALUES(?,?,?,?)",[  
(1,1,101,'Odd 2026'),  
(2,2,103,'Odd 2026'),  
(3,3,102,'Even 2026')  
])
```

```
query="""  
SELECT Student.Student_Name, Course.Course_Name  
FROM Enrollment
```

```
JOIN Student ON Enrollment.Student_ID=Student.Student_ID
```

```
JOIN Course ON Enrollment.Course_ID=Course.Course_ID
```

```
WHERE Semester='Odd 2026'
```

```
"""
```

```
cur.execute(query)
```

```
print("Student Name\tCourse")
```

```
for row in cur.fetchall():
```

```
    print(row[0],"\t",row[1])
```

```
conn.commit()
```

```
conn.close()
```

**o/p**

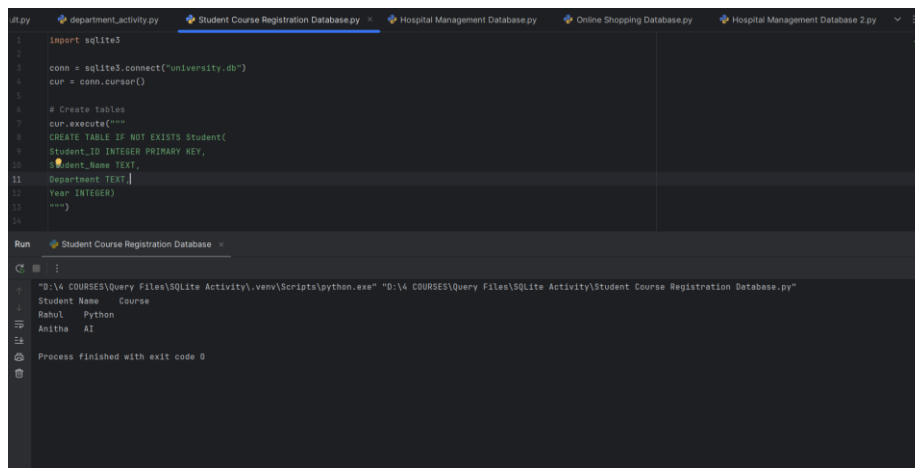
```
Student Name  Course
```

```
Rahul        Python
```

```
Anitha       AI
```

## Conclusion

The program successfully retrieved students and their enrolled courses for the Odd 2026 semester using SQL JOIN.



The screenshot shows a Python IDE with a file named 'Student Course Registration Database.py'. The code defines a SQLite database connection, creates a table named 'Student' with columns 'Student\_ID', 'Student\_Name', 'Department', and 'Year', and then executes a query to retrieve student names and their enrolled courses for the 'Odd 2026' semester. The output of the program is displayed in the 'Run' console, showing the following data:

| Student Name | Course |
|--------------|--------|
| Rahul        | Python |
| Anitha       | AI     |

The console also shows the message 'Process finished with exit code 0'.

# Question 2: Hospital Management Database

## Aim

To display patient names, doctor names, and appointment dates scheduled after **1 January 2026**.

## Theory

JOIN combines multiple tables, and the WHERE clause filters appointments after a specific date.

## CODE

```
import sqlite3
```

```
conn=sqlite3.connect("hospital.db")
```

```
cur=conn.cursor()
```

```
cur.execute("""
```

```
CREATE TABLE IF NOT EXISTS Doctor(
```

```
Doctor_ID INTEGER PRIMARY KEY,
```

```
Doctor_Name TEXT,
```

```
Specialization TEXT)
```

```
""")
```

```
cur.execute("""
```

```
CREATE TABLE IF NOT EXISTS Patient(
```

```
Patient_ID INTEGER PRIMARY KEY,
```

```
Patient_Name TEXT,
```

```
Age INTEGER)
```

```
""")
```

```
cur.execute("""
```

```
CREATE TABLE IF NOT EXISTS Appointment(  
Appointment_ID INTEGER PRIMARY KEY,  
Patient_ID INTEGER,  
Doctor_ID INTEGER,  
Appointment_Date TEXT,  
FOREIGN KEY(Patient_ID) REFERENCES Patient(Patient_ID),  
FOREIGN KEY(Doctor_ID) REFERENCES Doctor(Doctor_ID))  
""")
```

```
cur.execute("DELETE FROM Doctor")  
cur.execute("DELETE FROM Patient")  
cur.execute("DELETE FROM Appointment")
```

```
cur.executemany("INSERT INTO Doctor VALUES(?,?,?)",[  
(1,'Dr. Kumar','Cardiology'),  
(2,'Dr. Meena','Orthopedic')  
])
```

```
cur.executemany("INSERT INTO Patient VALUES(?,?,?)",[  
(1,'Ravi',35),  
(2,'Priya',28)  
])
```

```
cur.executemany("INSERT INTO Appointment VALUES(?,?,?,?)",[  
(1,1,1,'2026-02-10'),  
(2,2,2,'2025-12-20')  
])
```

```
query=""  
SELECT Patient.Patient_Name,  
Doctor.Doctor_Name,
```

```

Appointment.Appointment_Date
FROM Appointment
JOIN Patient ON Appointment.Patient_ID=Patient.Patient_ID
JOIN Doctor ON Appointment.Doctor_ID=Doctor.Doctor_ID
WHERE Appointment_Date>'2026-01-01'
"""

```

```

cur.execute(query)

print("Patient\tDoctor\tAppointment Date")

for row in cur.fetchall():

    print(row)

conn.commit()

conn.close()

```

**o/p:**

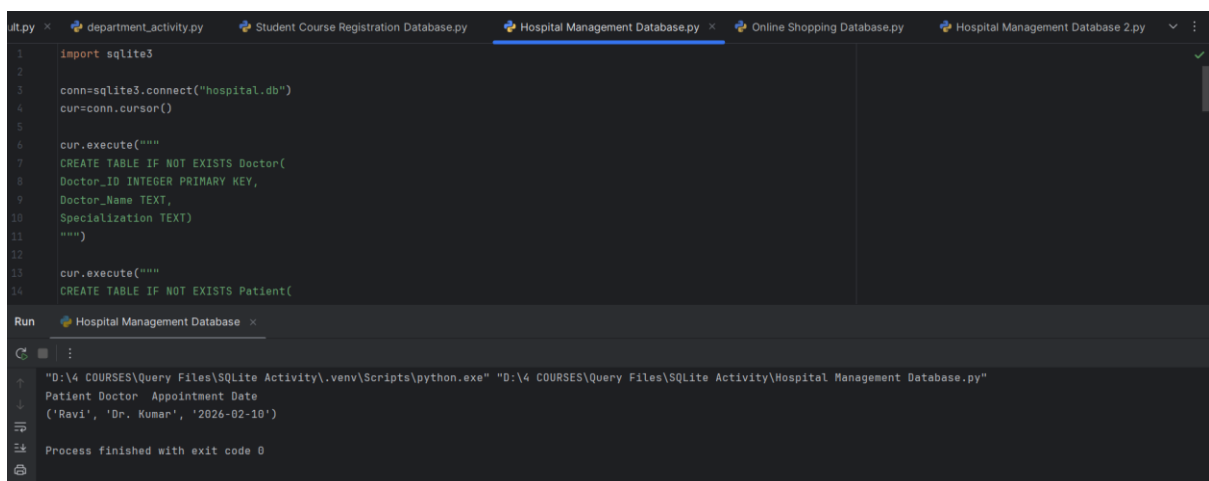
```

Patient   Doctor   Appointment Date
('Ravi', 'Dr. Kumar', '2026-02-10')

```

## Conclusion

The program successfully displayed appointments scheduled after 1 January 2026.



```

import sqlite3

conn=sqlite3.connect("hospital.db")
cur=conn.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS Doctor(
Doctor_ID INTEGER PRIMARY KEY,
Doctor_Name TEXT,
Specialization TEXT)
""")

cur.execute("""
CREATE TABLE IF NOT EXISTS Patient(
Patient_ID INTEGER PRIMARY KEY,
Patient_Name TEXT,
Appointment_Date TEXT)
""")

cur.execute("""
SELECT Patient.Patient_Name, Doctor.Doctor_Name, Appointment.Appointment_Date
FROM Appointment
JOIN Patient ON Appointment.Patient_ID=Patient.Patient_ID
JOIN Doctor ON Appointment.Doctor_ID=Doctor.Doctor_ID
WHERE Appointment_Date>'2026-01-01'
""")

for row in cur.fetchall():

    print(row)

conn.commit()

conn.close()

```

Run Hospital Management Database

```

"D:\4 COURSES\Query Files\SQLite Activity\.venv\Scripts\python.exe" "D:\4 COURSES\Query Files\SQLite Activity\Hospital Management Database.py"
Patient Doctor Appointment Date
('Ravi', 'Dr. Kumar', '2026-02-10')
Process finished with exit code 0

```

# Question 3: Online Shopping Database

## Aim

To retrieve customer name, product name, quantity, and order date using SQL JOIN.

## Theory

SQL JOIN combines Customer, Product, and Orders tables to display complete order information.

## CODE;

```
import sqlite3
```

```
conn=sqlite3.connect("shopping.db")
```

```
cur=conn.cursor()
```

```
cur.execute("""
```

```
CREATE TABLE IF NOT EXISTS Customer(
```

```
Customer_ID INTEGER PRIMARY KEY,
```

```
Customer_Name TEXT,
```

```
City TEXT)
```

```
""")
```

```
cur.execute("""
```

```
CREATE TABLE IF NOT EXISTS Product(
```

```
Product_ID INTEGER PRIMARY KEY,
```

```
Product_Name TEXT,
```

```
Price REAL)
```

```
""")
```

```
cur.execute("""
```

```
CREATE TABLE IF NOT EXISTS Orders(
```



```
Order_ID INTEGER PRIMARY KEY,  
Customer_ID INTEGER,  
Product_ID INTEGER,  
Quantity INTEGER,  
Order_Date TEXT,  
FOREIGN KEY(Customer_ID) REFERENCES Customer(Customer_ID),  
FOREIGN KEY(Product_ID) REFERENCES Product(Product_ID))  
""")
```

```
cur.execute("DELETE FROM Customer")  
cur.execute("DELETE FROM Product")  
cur.execute("DELETE FROM Orders")
```

```
cur.executemany("INSERT INTO Customer VALUES(?,?,?)",[  
(1,'Arun','Chennai'),  
(2,'Divya','Madurai')  
])
```

```
cur.executemany("INSERT INTO Product VALUES(?,?,?)",[  
(101,'Laptop',50000),  
(102,'Mouse',500)  
])
```

```
cur.executemany("INSERT INTO Orders VALUES(?,?,?,?,?)",[  
(1,1,101,1,'2026-03-15'),  
(2,2,102,2,'2026-04-01')  
])
```

```
query="""  
SELECT Customer.Customer_Name,  
Product.Product_Name,
```

```

Orders.Quantity,
Orders.Order_Date
FROM Orders
JOIN Customer ON Orders.Customer_ID=Customer.Customer_ID
JOIN Product ON Orders.Product_ID=Product.Product_ID
"""

```

```

cur.execute(query)

print("Customer\tProduct\tQuantity\tOrder Date")

for row in cur.fetchall():

    print(row)

conn.commit()

conn.close()

```

**o/p:**

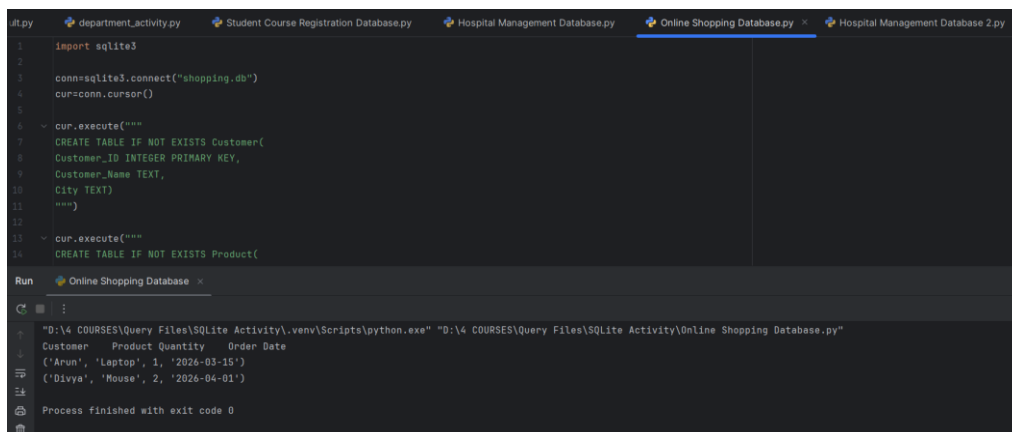
```

Customer  Product  Quantity  Order Date
('Arun', 'Laptop', 1, '2026-03-15')
('Divya', 'Mouse', 2, '2026-04-01')

```

## Conclusion

The program successfully retrieved customer order details using SQL JOIN.



```

import sqlite3

conn=sqlite3.connect("shopping.db")
cur=conn.cursor()

cur.execute("""
CREATE TABLE IF NOT EXISTS Customer(
Customer_ID INTEGER PRIMARY KEY,
Customer_Name TEXT,
City TEXT)
""")

cur.execute("""
CREATE TABLE IF NOT EXISTS Product(
Product_ID INTEGER PRIMARY KEY,
Product_Name TEXT,
Quantity INTEGER)
""")

cur.execute("""
INSERT INTO Customer (Customer_ID, Customer_Name, City)
VALUES (1, 'Arun', 'Laptop')
""")

cur.execute("""
INSERT INTO Product (Product_ID, Product_Name, Quantity)
VALUES (1, 'Laptop', 1), (2, 'Mouse', 2)
""")

conn.commit()

conn.close()

```

Run Online Shopping Database

```

"D:\4 COURSES\Query Files\SQLite Activity\.venv\Scripts\python.exe" "D:\4 COURSES\Query Files\SQLite Activity\Online Shopping Database.py"
Customer  Product Quantity  Order Date
('Arun', 'Laptop', 1, '2026-03-15')
('Divya', 'Mouse', 2, '2026-04-01')
Process finished with exit code 0

```

# Question 4: Hospital Management Database

## Aim

To retrieve doctor name, patient name, and appointment date for appointments scheduled during the current month.

## Theory

SQLite's `strftime()` function is used to compare the appointment month with the current month.

## CODE

```
import sqlite3

conn=sqlite3.connect("hospital.db")
cur=conn.cursor()

query="""
SELECT Doctor.Doctor_Name,
Patient.Patient_Name,
Appointment.Appointment_Date
FROM Appointment
JOIN Doctor ON Appointment.Doctor_ID=Doctor.Doctor_ID
JOIN Patient ON Appointment.Patient_ID=Patient.Patient_ID
WHERE strftime('%Y-%m',Appointment.Appointment_Date)=strftime('%Y-%m',now')
"""

cur.execute(query)

print("Doctor\tPatient\tAppointment Date")
rows=cur.fetchall()

if rows:
```

for row in rows:

print(row)

else:

print("No appointments found for the current month.")

conn.close()

**o/p**

Doctor Patient Appointment Date

No appointments found for the current month.

```
1 import sqlite3
2
3 conn=sqlite3.connect("hospital.db")
4 cur=conn.cursor()
5
6 query="""
7 SELECT Doctor.Doctor_Name,
8 Patient.Patient_Name,
9 Appointment.Appointment_Date
10 FROM Appointment
11 JOIN Doctor ON Appointment.Doctor_ID=Doctor.Doctor_ID
12 JOIN Patient ON Appointment.Patient_ID=Patient.Patient_ID
13 WHERE strftime('%Y-%m',Appointment.Appointment_Date)=strftime('%Y-%m', 'now')
14 """
15
16 Run Hospital Management Database 2 x
17
18 "D:\4 COURSES\Query Files\SQLite Activity\.venv\Scripts\python.exe" "D:\4 COURSES\Query Files\SQLite Activity\Hospital Management Database 2.py"
19 Doctor Patient Appointment Date
20 No appointments found for the current month.
21
22 Process finished with exit code 0
```

## Question 5: University Course Registration Database

### Aim

To retrieve student name, course name, and semester for courses having more than **3 credits**.

### Theory

JOIN combines Student, Course, and Enrollment tables, while the WHERE clause filters courses with credits greater than 3.

### Python Code

```
import sqlite3
```

```
conn=sqlite3.connect("university.db")
```

```
cur=conn.cursor()
```

```
query="""
```

```
SELECT Student.Student_Name,
```

```
Course.Course_Name,
```

```
Enrollment.Semester
```

```
FROM Enrollment
```

```
JOIN Student ON Enrollment.Student_ID=Student.Student_ID
```

```
JOIN Course ON Enrollment.Course_ID=Course.Course_ID
```

```
WHERE Course.Credits>3
```

```
"""
```

```
cur.execute(query)
```

```
print("Student\tCourse\tSemester")
```

```
for row in cur.fetchall():
```

```
    print(row)
```

```
conn.close()
```

**o/p:**

```
Student  Course  Semester
```

```
('Rahul', 'Python', 'Odd 2026')
```

```
('Anitha', 'AI', 'Odd 2026')
```

```
Course Registration Database.py  Hospital Management Database.py  Online Shopping Database.py  Hospital Management Database 2.py  Univer

1 import sqlite3
2
3 conn=sqlite3.connect("university.db")
4 cur=conn.cursor()
5
6 query="""
7 SELECT Student.Student_Name,
8 Course.Course_Name,
9 Enrollment.Semester
10 FROM Enrollment
11 JOIN Student ON Enrollment.Student_ID=Student.Student_ID
12 JOIN Course ON Enrollment.Course_ID=Course.Course_ID
13 WHERE Course.Credits>3
14 """
15
Run  University Course Registration Database x
"\"D:\4 COURSES\Query Files\SQLite Activity\venv\Scripts\python.exe\" \"D:\4 COURSES\Query Files\SQLite Activity\University Course Registr
Student Course Semester
('Rahul', 'Python', 'Odd 2026')
('Anitha', 'AI', 'Odd 2026')
Process finished with exit code 0
```

## Question 6: Banking Database

### Aim

To connect to an SQLite database and retrieve the customer name, account number, and balance for customers whose account balance exceeds **₹1,00,000**.

### Theory

SQLite stores relational data, and SQL JOIN is used to combine Customer and Account tables. The WHERE clause filters customers based on account balance.

### Python Code

```
import sqlite3
```

```
conn = sqlite3.connect("bank.db")
```

```
cur = conn.cursor()
```

```
# Create tables
```

```
cur.execute("""
```

```
CREATE TABLE IF NOT EXISTS Customer(
```

```
Customer_ID INTEGER PRIMARY KEY,
```

```
Customer_Name TEXT,
```

```
City TEXT)
```

```
""")
```

```
cur.execute("""
```

```
CREATE TABLE IF NOT EXISTS Account(
```

```
Account_No INTEGER PRIMARY KEY,
```

```
Customer_ID INTEGER,
```

```
Account_Type TEXT,
```

```
Balance REAL,
```

```
FOREIGN KEY(Customer_ID) REFERENCES Customer(Customer_ID))
```

```
""")
```

```
# Insert sample data
```

```
cur.execute("DELETE FROM Customer")
```

```
cur.execute("DELETE FROM Account")
```

```
cur.executemany("INSERT INTO Customer VALUES(?,?,?)",[
```

```
(1,'Arun','Chennai'),
```

```
(2,'Priya','Madurai'),
```

```
(3,'Rahul','Coimbatore')
```

```
])
```

```
cur.executemany("INSERT INTO Account VALUES(?,?,?,?)",[
```

```
(1001,1,'Savings',150000),
```

```
(1002,2,'Current',80000),
```

```
(1003,3,'Savings',250000)
```

```
])
```

```
query = ""
```

```
SELECT Customer.Customer_Name,
```

```
Account.Account_No,
```

```
Account.Balance
```

FROM Customer

JOIN Account ON Customer.Customer\_ID=Account.Customer\_ID

WHERE Account.Balance>100000

"""

cur.execute(query)

print("Customer\tAccount No\tBalance")

for row in cur.fetchall():

print(row)

conn.commit()

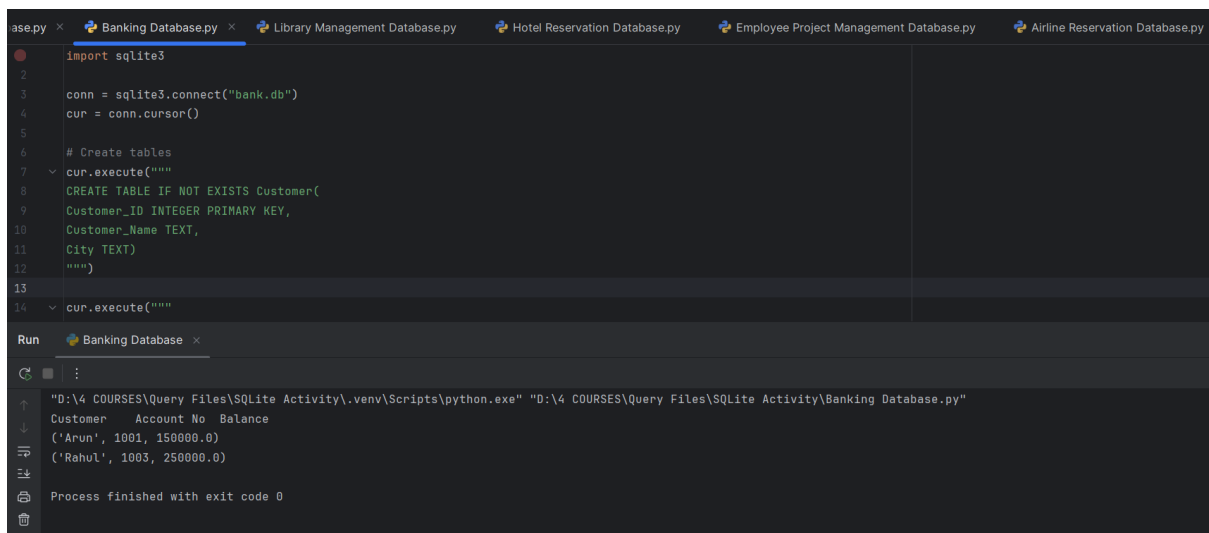
conn.close()

**O/P**

Customer Account No Balance

('Arun', 1001, 150000.0)

('Rahul', 1003, 250000.0)



The screenshot shows a Python IDE with several tabs. The active tab is 'Banking Database.py'. The code in the editor is as follows:

```
1 import sqlite3
2
3 conn = sqlite3.connect("bank.db")
4 cur = conn.cursor()
5
6 # Create tables
7 cur.execute("""
8 CREATE TABLE IF NOT EXISTS Customer(
9 Customer_ID INTEGER PRIMARY KEY,
10 Customer_Name TEXT,
11 City TEXT)
12 """)
13
14 cur.execute("""
```

Below the editor, the 'Run' output is displayed. It shows the command executed and the output of the script:

```
"D:\4 COURSES\Query Files\SQLite Activity\.venv\Scripts\python.exe" "D:\4 COURSES\Query Files\SQLite Activity\Banking Database.py"
Customer Account No Balance
('Arun', 1001, 150000.0)
('Rahul', 1003, 250000.0)
Process finished with exit code 0
```



# Question 7: Library Management Database

## Aim

To retrieve the member name, book title, and issue date for books issued after **1 January 2026**.

## Theory

SQL JOIN combines Member, Book, and Issue tables, while the WHERE clause filters issue dates.

## Python Code

```
import sqlite3
```

```
conn = sqlite3.connect("library.db")
```

```
cur = conn.cursor()
```

```
cur.execute("""
CREATE TABLE IF NOT EXISTS Member(
Member_ID INTEGER PRIMARY KEY,
Member_Name TEXT,
Phone TEXT)
""")
```

```
cur.execute("""
CREATE TABLE IF NOT EXISTS Book(
Book_ID INTEGER PRIMARY KEY,
Book_Title TEXT,
Author TEXT)
""")
```

```
cur.execute("""
CREATE TABLE IF NOT EXISTS IssueBook(
```

```
Issue_ID INTEGER PRIMARY KEY,  
Member_ID INTEGER,  
Book_ID INTEGER,  
Issue_Date TEXT,  
FOREIGN KEY(Member_ID) REFERENCES Member(Member_ID),  
FOREIGN KEY(Book_ID) REFERENCES Book(Book_ID))  
""")
```

```
cur.execute("DELETE FROM Member")  
cur.execute("DELETE FROM Book")  
cur.execute("DELETE FROM IssueBook")
```

```
cur.executemany("INSERT INTO Member VALUES(?,?,?)",[  
(1,'Kumar','9876543210'),  
(2,'Anitha','9876501234')  
])
```

```
cur.executemany("INSERT INTO Book VALUES(?,?,?)",[  
(101,'Python Basics','John'),  
(102,'DBMS Concepts','James')  
])
```

```
cur.executemany("INSERT INTO IssueBook VALUES(?,?,?,?)",[  
(1,1,101,'2026-02-10'),  
(2,2,102,'2025-12-15')  
])
```

```
query="""  
SELECT Member.Member_Name,  
Book.Book_Title,  
IssueBook.Issue_Date
```

```

FROM IssueBook

JOIN Member ON IssueBook.Member_ID=Member.Member_ID

JOIN Book ON IssueBook.Book_ID=Book.Book_ID

WHERE Issue_Date>'2026-01-01'

"""

cur.execute(query)


print("Member\tBook\tIssue Date")

for row in cur.fetchall():

    print(row)


conn.commit()

conn.close()

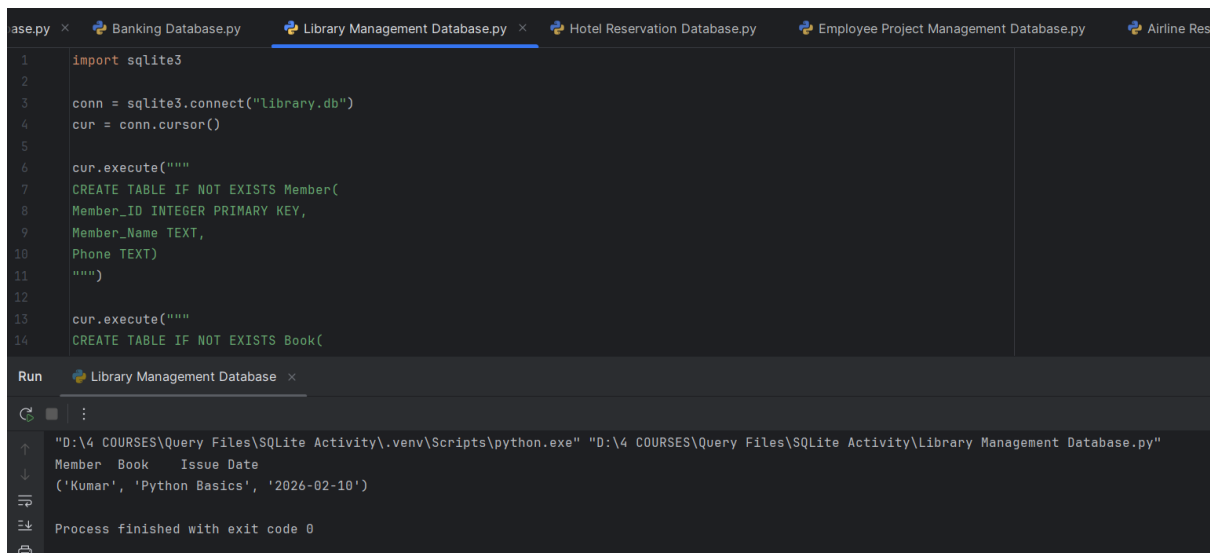
```

**O/P:**

```

Member    Book      Issue Date
('Kumar', 'Python Basics', '2026-02-10')

```



```

1 import sqlite3
2
3 conn = sqlite3.connect("library.db")
4 cur = conn.cursor()
5
6 cur.execute("""
7 CREATE TABLE IF NOT EXISTS Member(
8 Member_ID INTEGER PRIMARY KEY,
9 Member_Name TEXT,
10 Phone TEXT)
11 """)
12
13 cur.execute("""
14 CREATE TABLE IF NOT EXISTS Book(
15 Book_ID INTEGER PRIMARY KEY,
16 Book_Name TEXT,
17 Issue_Date TEXT)
18 """)
19
20 cur.execute("""
21 INSERT INTO Book(Book_ID, Book_Name, Issue_Date)
22 VALUES(1, 'Python Basics', '2026-02-10')
23 """)
24
25 conn.commit()
26
27 conn.close()

```

Run Library Management Database

```

"D:\4 COURSES\Query Files\SQLite Activity\.venv\Scripts\python.exe" "D:\4 COURSES\Query Files\SQLite Activity\Library Management Database.py"
Member Book Issue Date
('Kumar', 'Python Basics', '2026-02-10')
Process finished with exit code 0

```

## Question 8: Hotel Reservation Database

## **Aim**

To retrieve the customer name, room type, and check-in date for customers who reserved **Deluxe** rooms.

## **Theory**

JOIN combines Customer, Room, and Reservation tables, while WHERE filters Deluxe room reservations.

## **Python Code**

```
import sqlite3
```

```
conn=sqlite3.connect("hotel.db")
```

```
cur=conn.cursor()
```

```
cur.execute("""
```

```
CREATE TABLE IF NOT EXISTS Customer(
```

```
Customer_ID INTEGER PRIMARY KEY,
```

```
Customer_Name TEXT,
```

```
City TEXT)
```

```
""")
```

```
cur.execute("""
```

```
CREATE TABLE IF NOT EXISTS Room(
```

```
Room_ID INTEGER PRIMARY KEY,
```

```
Room_Type TEXT,
```

```
Rent REAL)
```

```
""")
```

```
cur.execute("""
```

```
CREATE TABLE IF NOT EXISTS Reservation(
```

```
Reservation_ID INTEGER PRIMARY KEY,
```

```
Customer_ID INTEGER,
```

```
Room_ID INTEGER,
```

```
Check_In_Date TEXT,
```

```
Check_Out_Date TEXT,
```

```
FOREIGN KEY(Customer_ID) REFERENCES Customer(Customer_ID),
```

```
FOREIGN KEY(Room_ID) REFERENCES Room(Room_ID))
```

```
""")
```

```
cur.execute("DELETE FROM Customer")
```

```
cur.execute("DELETE FROM Room")
```

```
cur.execute("DELETE FROM Reservation")
```

```
cur.executemany("INSERT INTO Customer VALUES(?,?,?)",[
```

```
(1,'Ravi','Chennai'),
```

```
(2,'Divya','Madurai')
```

```
])
```

```
cur.executemany("INSERT INTO Room VALUES(?,?,?)",[
```

```
(101,'Deluxe',3500),
```

```
(102,'Standard',2000)
```

)

```
cur.executemany("INSERT INTO Reservation VALUES(?,?,?,?)",[  
(1,1,101,'2026-08-10','2026-08-12'),  
(2,2,102,'2026-08-15','2026-08-16')
```

)

```
query="""
```

```
SELECT Customer.Customer_Name,
```

```
Room.Room_Type,
```

```
Reservation.Check_In_Date
```

```
FROM Reservation
```

```
JOIN Customer ON Reservation.Customer_ID=Customer.Customer_ID
```

```
JOIN Room ON Reservation.Room_ID=Room.Room_ID
```

```
WHERE Room.Room_Type='Deluxe'
```

```
"""
```

```
cur.execute(query)
```

```
print("Customer\tRoom Type\tCheck-In")
```

```
for row in cur.fetchall():
```

```
    print(row)
```

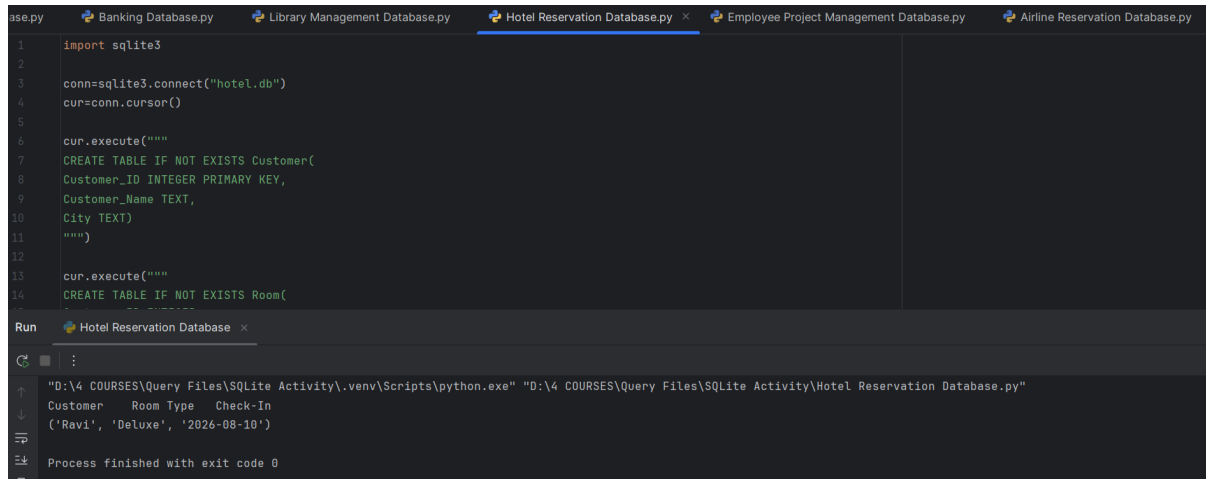
```
conn.commit()
```

```
conn.close()
```

O/P

Customer    Room Type    Check-In

('Ravi', 'Deluxe', '2026-08-10')



```
1 import sqlite3
2
3 conn=sqlite3.connect("hotel.db")
4 cur=conn.cursor()
5
6 cur.execute("""
7 CREATE TABLE IF NOT EXISTS Customer(
8 Customer_ID INTEGER PRIMARY KEY,
9 Customer_Name TEXT,
10 City TEXT)
11 """)
12
13 cur.execute("""
14 CREATE TABLE IF NOT EXISTS Room(
15 Room_ID INTEGER PRIMARY KEY,
16 Room_Type TEXT,
17 Check-In TEXT)
18 """)
19
20 cur.execute("""
21 INSERT INTO Customer (Customer_ID, Customer_Name, City)
22 VALUES (1, 'Ravi', 'Deluxe')
23 """)
24
25 conn.close()
```

Run Hotel Reservation Database x

"D:\4 COURSES\Query Files\SQLite Activity\.venv\Scripts\python.exe" "D:\4 COURSES\Query Files\SQLite Activity\Hotel Reservation Database.py"

Customer    Room Type    Check-In

('Ravi', 'Deluxe', '2026-08-10')

Process finished with exit code 0

## Question 9: Employee Project Management Database

### Aim

To retrieve the employee name, project name, and role for projects having a budget greater than ₹50,00,000.

### Theory

JOIN combines Employee, Project, and Works\_On tables. The WHERE clause filters projects based on budget.

### Python Code

```
import sqlite3
```

```
conn=sqlite3.connect("company.db")
```

```
cur=conn.cursor()
```

```
cur.execute("""  
  
CREATE TABLE IF NOT EXISTS Employee(  
  
Employee_ID INTEGER PRIMARY KEY,  
  
Employee_Name TEXT,  
  
Department TEXT)  
""")
```

```
cur.execute("""  
  
CREATE TABLE IF NOT EXISTS Project(  
  
Project_ID INTEGER PRIMARY KEY,  
  
Project_Name TEXT,  
  
Budget REAL)  
""")
```

```
cur.execute("""  
  
CREATE TABLE IF NOT EXISTS Works_On(  
  
Employee_ID INTEGER,  
  
Project_ID INTEGER,  
  
Role TEXT,  
  
FOREIGN KEY(Employee_ID) REFERENCES Employee(Employee_ID),  
  
FOREIGN KEY(Project_ID) REFERENCES Project(Project_ID))  
""")
```



```
cur.execute("DELETE FROM Employee")
```

```
cur.execute("DELETE FROM Project")
```

```
cur.execute("DELETE FROM Works_On")
```

```
cur.executemany("INSERT INTO Employee VALUES(?,?,?)",[
```

```
(1,'Ajay','Development'),
```

```
(2,'Sneha','Testing')
```

```
])
```

```
cur.executemany("INSERT INTO Project VALUES(?,?,?)",[
```

```
(101,'ERP System',6000000),
```

```
(102,'Website',3000000)
```

```
])
```

```
cur.executemany("INSERT INTO Works_On VALUES(?,?,?)",[
```

```
(1,101,'Developer'),
```

```
(2,102,'Tester')
```

```
])
```

```
query="""
```

```
SELECT Employee.Employee_Name,
```

```
Project.Project_Name,
```

```
Works_On.Role
```

```
FROM Works_On
```

JOIN Employee ON Works\_On.Employee\_ID=Employee.Employee\_ID

JOIN Project ON Works\_On.Project\_ID=Project.Project\_ID

WHERE Project.Budget>5000000

"""

cur.execute(query)

print("Employee\tProject\tRole")

for row in cur.fetchall():

print(row)

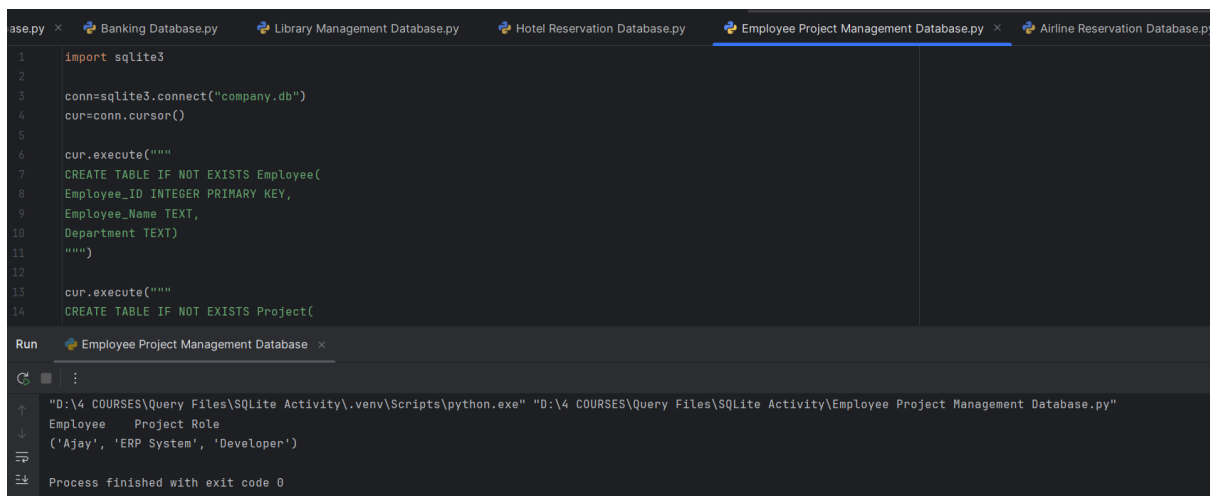
conn.commit()

conn.close()

O/P

Employee    Project        Role

('Ajay', 'ERP System', 'Developer')



```
1 import sqlite3
2
3 conn=sqlite3.connect("company.db")
4 cur=conn.cursor()
5
6 cur.execute("""
7 CREATE TABLE IF NOT EXISTS Employee(
8 Employee_ID INTEGER PRIMARY KEY,
9 Employee_Name TEXT,
10 Department TEXT)
11 """)
12
13 cur.execute("""
14 CREATE TABLE IF NOT EXISTS Project(
15 Project_ID INTEGER PRIMARY KEY,
16 Project_Name TEXT,
17 Budget INTEGER)
18 """)
19
20 conn.commit()
21 conn.close()
22
23 print("Employee\tProject\tRole")
24 for row in cur.fetchall():
25     print(row)
```

Run Employee Project Management Database

"D:\4 COURSES\Query Files\SQLite Activity\.venv\Scripts\python.exe" "D:\4 COURSES\Query Files\SQLite Activity\Employee Project Management Database.py"

Employee    Project Role  
(Ajay, ERP System, Developer)

Process finished with exit code 0

# Question 10: Airline Reservation Database

## Aim

To retrieve the passenger name, flight name, destination, and travel date for passengers travelling to **Delhi**.

## Theory

JOIN combines Passenger, Flight, and Booking tables. The WHERE clause filters flights with the destination "Delhi."

## Python Code

```
import sqlite3
```

```
conn=sqlite3.connect("airline.db")
```

```
cur=conn.cursor()
```

```
cur.execute("""
```

```
CREATE TABLE IF NOT EXISTS Passenger(
```

```
Passenger_ID INTEGER PRIMARY KEY,
```

```
Passenger_Name TEXT,
```

```
City TEXT)
```

```
""")
```

```
cur.execute("""
```

```
CREATE TABLE IF NOT EXISTS Flight(
```

```
Flight_ID INTEGER PRIMARY KEY,
```

```
Flight_Name TEXT,
```

Destination TEXT)

""")

cur.execute("""

CREATE TABLE IF NOT EXISTS Booking(

Booking\_ID INTEGER PRIMARY KEY,

Passenger\_ID INTEGER,

Flight\_ID INTEGER,

Travel\_Date TEXT,

FOREIGN KEY(Passenger\_ID) REFERENCES Passenger(Passenger\_ID),

FOREIGN KEY(Flight\_ID) REFERENCES Flight(Flight\_ID))

""")

cur.execute("DELETE FROM Passenger")

cur.execute("DELETE FROM Flight")

cur.execute("DELETE FROM Booking")

cur.executemany("INSERT INTO Passenger VALUES(?,?,?)",[

(1,'Vijay','Chennai'),

(2,'Meera','Madurai')

])

cur.executemany("INSERT INTO Flight VALUES(?,?,?)",[

(101,'IndiGo 6E101','Delhi'),

```
(102,'Air India AI202','Mumbai')
```

```
]
```

```
cur.executemany("INSERT INTO Booking VALUES(?,?,?,?)",[
```

```
(1,1,101,'2026-09-01'),
```

```
(2,2,102,'2026-09-03')
```

```
]
```

```
query="""
```

```
SELECT Passenger.Passenger_Name,
```

```
Flight.Flight_Name,
```

```
Flight.Destination,
```

```
Booking.Travel_Date
```

```
FROM Booking
```

```
JOIN Passenger ON Booking.Passenger_ID=Passenger.Passenger_ID
```

```
JOIN Flight ON Booking.Flight_ID=Flight.Flight_ID
```

```
WHERE Flight.Destination='Delhi'
```

```
"""
```

```
cur.execute(query)
```

```
print("Passenger\tFlight\tDestination\tTravel Date")
```

```
for row in cur.fetchall():
```

```
    print(row)
```

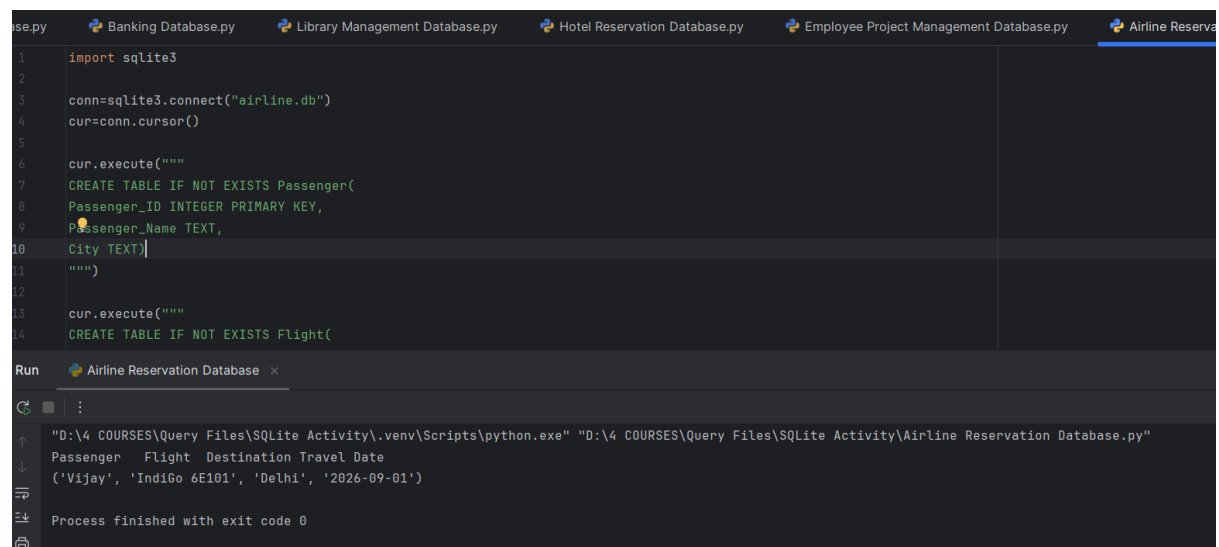
conn.commit()

conn.close()

O/P

| Passenger | Flight | Destination | Travel Date |
|-----------|--------|-------------|-------------|
|-----------|--------|-------------|-------------|

|           |                 |          |               |
|-----------|-----------------|----------|---------------|
| ('Vijay', | 'IndiGo 6E101', | 'Delhi', | '2026-09-01') |
|-----------|-----------------|----------|---------------|



The screenshot shows a Python IDE with a dark theme. The top bar displays several open files: 'se.py', 'Banking Database.py', 'Library Management Database.py', 'Hotel Reservation Database.py', 'Employee Project Management Database.py', and 'Airline Reserva...'. The main editor window shows a Python script with the following code:

```
1 import sqlite3
2
3 conn=sqlite3.connect("airLine.db")
4 cur=conn.cursor()
5
6 cur.execute("""
7 CREATE TABLE IF NOT EXISTS Passenger(
8 Passenger_ID INTEGER PRIMARY KEY,
9 Passenger_Name TEXT,
10 City TEXT)|
11 """)
12
13 cur.execute("""
14 CREATE TABLE IF NOT EXISTS Flight(
```

Below the editor, there is a 'Run' button and a terminal window. The terminal output shows the command executed and the resulting data:

```
"D:\4 COURSES\Query Files\SQLite Activity\venv\Scripts\python.exe" "D:\4 COURSES\Query Files\SQLite Activity\Airline Reservation Database.py"
Passenger Flight Destination Travel Date
('Vijay', 'IndiGo 6E101', 'Delhi', '2026-09-01')
Process finished with exit code 0
```